

淘宝直播 AI agent前端开发工程师

一面（技术基础） · 1-6

1. 请简述一下 SSE 和 WebSocket 的区别，在 AI 流式输出场景下你会如何选择？
2. 在处理 AI 响应的长列表消息时，如何实现自动滚动到底部，并保证用户手动向上滚动时不被干扰？
3. 请手写一个具有并发限制的请求调度器，模拟同时调用多个 AI 接口的场景。
4. 谈谈你对 CSS 容器查询（Container Queries）的理解，它在 AI 侧边栏组件中有什么作用？
5. 如何实现一个简单的 Markdown 实时渲染引擎，并处理其中的代码高亮？
6. 解释一下 JS 中的事件循环（Event Loop），并说明宏任务与微任务在处理 AI 消息推送时的先后顺序。

二面（深入原理与项目） · 1-8

1. AI 流式输出时，前端如何解决“打字机”效果带来的频繁 DOM 渲染导致的性能问题？
 1. 在开发 AI Agent 时，前端如何管理复杂的 Prompt 模板以及变量替换逻辑？
 2. 谈谈 Function Calling（函数调用）的前端实现流程，前端需要做哪些配合？
 3. 如何处理 AI 响应过程中的网络中断或内容截断？请设计一套容错机制。
 4. 在直播间这种高频交互环境下，AI 组件如何与直播间原有的状态（如点赞、弹幕）进行高效同步？
 5. 谈谈 React Fiber 架构对 AI 聊天机器人这类长文本更新场景的意义。
 6. 如果 AI 返回的内容包含敏感信息被后端拦截，前端应该如何优雅地展示和处理？
 7. 请解释 Web Worker 在 AI 前端开发中的应用场景，你会用它处理什么？

三面（架构与综合能力） 1-5

1. 如果让你从零设计一个淘宝直播 AI Agent 的前端架构，你会重点考虑哪些模块？
2. 在 AI 时代，你认为前端工程师的核心竞争力发生了什么变化？
3. 如何衡量一个 AI Agent 前端产品的质量？你会关注哪些核心指标？
4. 聊聊你在项目中遇到的最难的一个 AI 相关技术挑战，你是如何解决的？
5. AI 生成的内容具有不确定性，前端如何通过设计手段降低这种不确定性带来的负面体验？

一面（技术基础）

1. 请简述一下 SSE 和 WebSocket 的区别，在 AI 流式输出场景下你会如何选择？

难度：★★

考点：网络协议、流式传输、AI 交互基础

答案要点：

SSE 是基于 HTTP 的单向通信协议，而 WebSocket 是全双工的独立协议，在 AI 问答场景中 SSE 往往因其轻量和自动重连特性更受青睐。

- 协议层：SSE 运行在 HTTP 之上，支持自动重连；WebSocket 是独立协议，需要处理握手和心跳。
- 数据流向：SSE 仅支持服务端向客户端推送；WebSocket 支持双向实时通信。
- 浏览器支持：SSE 原生支持 TextStream 处理，非常适合 LLM 的逐字生成效果。
- 复杂性：SSE 实现简单，对现有 HTTP 架构兼容性好，无需额外配置网关支持。

常见坑：

- 忽略了 HTTP/1.1 下浏览器对 SSE 有 6 个并发连接的限制。
- 忘记在服务端设置 Cache-Control: no-cache。

追问：

- 如果需要客户端在传输过程中中断请求，SSE 应该如何操作？

2. 在处理 AI 响应的长列表消息时，如何实现自动滚动到底部，并保证用户手动向上滚动时不被干扰？

难度：★★

考点：DOM 操作、滚动原理、用户体验优化

答案要点：

通过监听容器的 scroll 事件判断用户是否处于底部，仅在处于底部时随新内容自动滚动。

- 判定逻辑：通过 $scrollHeight - scrollTop === clientHeight$ 判断是否触底。
- 阈值处理：通常会预留 10-20px 的偏差量，防止因浮点数计算导致的判定失败。
- 行为控制：使用 `scrollIntoView({ behavior: 'smooth' })` 或直接修改 `scrollTop`。
- 状态锁：当用户手动向上滚动超过一定距离时，关闭自动滚动开关。

常见坑：

- 在 React/Vue 中更新数据后立即操作 DOM，导致获取的 `offsetHeight` 是旧的。
- 频繁触发滚动事件导致性能抖动。

追问：

- 如果消息中包含图片，图片加载延迟会导致高度变化，你如何处理？

3. 请手写一个具有并发限制的请求调度器，模拟同时调用多个 AI 接口的场景。

难度：★★★

考点：Promise 异步控制、队列数据结构

答案要点：

该调度器需要维护一个执行队列和一个等待队列，确保当前运行的任务数不超过预设的 limit。

- 核心结构：内部维护 count 计数器和 queue 数组。
- 任务入队：每次 add 任务时判断 count 是否小于 limit。
- 递归执行：任务完成后，在 finally 回调中取出队首任务继续执行，直到队列清空。
- 结果返回：add 方法应返回一个 Promise，以便外部获取单个任务的结果。

常见坑：

- 忘记处理任务执行失败的情况，导致计数器无法重置，后续任务阻塞。
- 没有返回 Promise，导致调用方无法 await 结果。

追问：

- 如何给这个调度器增加一个超时自动取消的功能？

代码块

```
1  class TaskQueue {
2    constructor(limit) {
3      this.limit = limit;
4      this.count = 0;
5      this.queue = [];
6    }
7    async add(fn) {
8      if (this.count >= this.limit) {
9        await new Promise(resolve => this.queue.push(resolve));
10     }
11     this.count++;
12     try {
13       return await fn();
14     } finally {
15       this.count--;
16       if (this.queue.length) this.queue.shift()();
17     }
18   }
19 }
```

4. 谈谈你对 CSS 容器查询（Container Queries）的理解，它在 AI 侧边栏组件中有什么作用？

难度：★★

考点：CSS 布局、响应式设计

答案要点：

容器查询允许组件根据其父容器的大小而非浏览器视口的大小来改变样式，这在复杂的 AI 插件场景中非常有用。

- 解决痛点：AI Agent 经常以侧边栏、浮窗或嵌入式组件形式存在，视口宽度无法反映其实际空间。
- 语法实现：使用 container-type 声明容器，通过 @container 编写样式。
- 灵活性：同一个 AI 聊天组件，在窄边栏显示紧凑模式，在宽容器显示完整模式。
- 性能：比监听 ResizeObserver 并动态修改 class 的性能更高。

常见坑：

- 忘记给父元素设置 container-type。
- 浏览器兼容性问题，在旧版内核中需要 polyfill。

追问：

- 容器查询和媒体查询 (Media Queries) 可以同时使用吗?

5. 如何实现一个简单的 Markdown 实时渲染引擎，并处理其中的代码高亮？

难度：★★

考点：字符串处理、第三方库集成、安全性

答案要点:

通常使用 `marked` 或 `markdown-it` 配合 `highlight.js` 或 `prismjs` 实现，重点在于流式输出时的增量渲染。

- 解析流程：将 LLM 返回的 Markdown 字符串传入解析器转为 HTML。
- 代码高亮：在解析器的渲染钩子（renderer）中调用高亮库。
- XSS 防护：必须使用 DOMPurify 等工具对生成的 HTML 进行清洗。
- 性能优化：对于超长文本，避免全量重新渲染，采用分段解析或增量更新。

常见坑：

- 直接使用 innerHTML 注入内容导致 XSS 攻击。
- 流式输出过程中，Markdown 语法标记（如 `` ` ``）被截断导致渲染乱码。

追问：

- 如果 Markdown 里有数学公式 (LaTeX)，你会怎么处理？

6. 解释一下 JS 中的事件循环（Event Loop），并说明宏任务与微任务在处理 AI 消息推送时的先后顺序。

难度：★

考点：JS 执行机制、异步原理

答案要点:

事件循环是 JS 处理异步任务的核心机制，微任务会在当前宏任务执行完后、渲染前清空。

- 执行顺序：同步代码 -> 微任务（Promise.then, MutationObserver） -> 渲染 -> 宏任务（setTimeout, I/O）。
- AI 场景：从 SSE 接收到的数据回调属于宏任务，而处理数据状态更新的 Promise 属于微任务。

- 渲染时机：微任务执行完后浏览器会尝试进行 UI 渲染，因此在微任务中频繁更新状态可能导致掉帧。

常见坑：

- 误以为 Promise 构造函数里的代码是异步的。
- 在微任务中递归创建微任务导致页面卡死。

追问：

- requestAnimationFrame 处于事件循环的哪个阶段？
-

二面（深入原理与项目）

1. AI 流式输出时，前端如何解决“打字机”效果带来的频繁 DOM 渲染导致的性能问题？

难度：★★★

考点：性能优化、渲染策略、虚拟 DOM

答案要点：

核心思路是降低渲染频率，通过缓冲区（Buffer）或节流机制来合并更新。

- 缓冲区机制：将 SSE 接收到的字符先存入 Buffer，每隔 50-100ms 批量更新一次状态。
- 增量更新：仅对当前正在生成的最后一条消息进行局部更新，避免全量列表 Diff。
- 样式优化：使用 contain: content 等 CSS 属性限制重绘范围。
- 动画处理：打字机光标使用 CSS 动画而非 JS 模拟，减少主线程压力。

常见坑：

- 每次收到一个字符都触发一次 setState，导致 React 频繁执行 Fiber 调和。
- 忽略了长文本下的内存占用问题。

追问：

- 如果一条消息有几万字，虚拟列表（Virtual List）能解决这个问题吗？

2. 在开发 AI Agent 时，前端如何管理复杂的 Prompt 模板以及变量替换逻辑？

难度：★★

考点：工程化、逻辑抽象

答案要点：

应当将 Prompt 与业务代码解耦，建立一套标准化的模板管理系统。

- 模板定义：使用类似 Mustache 的占位符语法 {{variable}} 定义模板。
- 动态注入：封装工具函数，自动从当前上下文（如用户信息、直播间状态）中提取变量并填充。
- 版本控制：前端维护一套 Prompt 字典，支持根据不同的 Agent ID 加载不同的配置。

- 安全过滤：在填充变量前进行敏感词过滤或格式化，防止 Prompt 注入。

常见坑：

- 将长篇大论的 Prompt 直接硬编码在组件内部。
- 变量替换时没有处理特殊字符，导致 LLM 解析异常。

追问：

- 如何在前端实现 Prompt 的 A/B 测试？

3. 谈谈 Function Calling（函数调用）的前端实现流程，前端需要做哪些配合？

难度：★★★

考点：AI Agent 交互协议、前后端协作

答案要点：

前端主要负责定义工具 Schema、解析模型返回的调用指令并执行本地逻辑。

- Schema 定义：按照 OpenAI 标准编写 JSON Schema，描述函数名、参数类型及描述。
- 指令拦截：当模型返回 `finish_reason: 'tool_calls'` 时，前端解析出 `function_name` 和 `arguments`。
- 本地执行：在前端执行对应的逻辑（如：查询直播间商品、跳转页面），并将结果再次发给模型。
- 状态反馈：在执行本地函数时，UI 应展示“正在查询...”等中间状态。

常见坑：

- 模型生成的 JSON 参数不合法，前端解析报错。
- 函数执行时间过长导致对话链路超时。

追问：

- 如果模型一次性返回了多个函数调用请求，你应该如何处理？

4. 如何处理 AI 响应过程中的网络中断或内容截断？请设计一套容错机制。

难度：★★

考点：异常处理、健壮性设计

答案要点：

通过状态机管理对话状态，结合断线重连和手动重试机制确保链路闭环。

- 状态标记：为每条消息设置 `loading`、`error`、`success`、`truncated` 等状态。
- 自动重连：SSE 连接断开时，记录当前已接收的 `tokens` 数，尝试断点续传（需后端支持）。
- 交互补偿：提供“重新生成”按钮，并允许用户手动编辑上一条发送的消息。
- 内容检测：通过解析 Markdown 标签是否闭合来判断内容是否被截断，并给出提示。

常见坑：

- 接口报错后，UI 一直停留在 `loading` 状态。

- 重试时没有清理旧的缓冲区数据，导致内容重叠。

追问：

- 如何在前端监控 AI 响应的“首字延迟”（TTFT）？

5. 在直播间这种高频交互环境下，AI 组件如何与直播间原有的状态（如点赞、弹幕）进行高效同步？

难度：★★★

考点：状态管理、跨模块通信

答案要点：

采用发布订阅模式或全局状态管理，将 AI Agent 作为一个独立的观察者接入直播流。

- 消息总线：AI 组件订阅直播间的 EventBus，实时获取弹幕、礼物等事件。
- 优先级队列：对高频事件进行削峰填谷，避免短时间内触发过多次 AI 逻辑。
- 状态共享：使用 Zustand 或 Redux 存储直播间元数据，Agent 随时读取作为上下文。
- 渲染隔离：AI 动画与直播视频流分层渲染，确保 AI 的交互不影响视频播放的流畅度。

常见坑：

- 弹幕过多导致 AI 组件频繁重绘，造成直播间卡顿。
- 状态同步不及时，导致 AI 说的内容与直播间当前画面脱节。

追问：

- 如果要实现“AI 自动回复弹幕”，你会如何设计前端的过滤逻辑？

6. 谈谈 React Fiber 架构对 AI 聊天机器人这类长文本更新场景的意义。

难度：★★★

考点：React 原理、调度算法

答案要点：

Fiber 通过时间分片（Time Slicing）将渲染工作拆分为小单元，防止长文本更新阻塞主线程。

- 任务拆分：在 Diff 阶段，Fiber 允许中断和恢复，确保高优先级的用户输入能及时响应。
- 优先级调度：AI 字符更新可以设置为较低优先级，而用户点击、滚动设置为高优先级。
- 并发模式：利用 useTransition 等 Hook，可以让 AI 文本的渲染在后台进行，不卡顿 UI。
- 稳定性：避免了旧版 React 递归 Diff 导致的调用栈过深问题。

常见坑：

- 误以为用了 React 18 性能就一定好，却在 render 函数里写了大量耗时计算。
- 滥用 memo 导致不必要的内存开销。

追问：

- useDeferredValue 在 AI 场景下有什么具体用途？

7. 如果 AI 返回的内容包含敏感信息被后端拦截，前端应该如何优雅地展示和处理？

难度：★★

考点：内容安全、用户体验

答案要点：

前端需要配合后端的安全策略，通过特定的错误码或信令进行 UI 降级处理。

- 局部遮蔽：对已渲染的部分保留，后续违规部分显示为星号或提示占位符。
- 引导话术：展示预设的合规提示语，而非冷冰冰的“接口报错”。
- 状态回滚：如果整条消息违规，将其从对话历史中移除或标记为不可见。
- 上报机制：自动记录违规上下文并上报，辅助优化后端的过滤模型。

常见坑：

- 直接在控制台打印后端返回的完整错误信息，可能泄露敏感逻辑。
- 拦截太生硬，导致用户以为是程序 Bug。

追问：

- 如何防止用户通过前端手段绕过安全过滤？

8. 请解释 Web Worker 在 AI 前端开发中的应用场景，你会用它处理什么？

难度：★★

考点：多线程、计算密集型任务

答案要点：

Web Worker 用于将非 UI 相关的重计算逻辑移出主线程，保持页面响应。

- 文本分词：在前端进行 RAG 相关的文本分词（Tokenization）或向量化预处理。
- 复杂解析：对 LLM 返回的大规模 JSON 或复杂的逻辑树进行解析。
- 离线模型：如果运行轻量级的 WebLLM，必须在 Worker 中执行以防卡死。
- 埋点计算：对用户行为数据进行实时分析和聚合。

常见坑：

- 在 Worker 中尝试操作 DOM。
- 频繁在主线程和 Worker 之间传递大数据量导致序列化开销过大。

追问：

- 除了 postMessage，还有什么方式可以实现主线程与 Worker 的数据共享？

三面（架构与综合能力）

1. 如果让你从零设计一个淘宝直播 AI Agent 的前端架构，你会重点考虑哪些模块？

难度：★★★

考点：系统架构、模块化设计

答案要点：

一个成熟的 Agent 架构应包含连接层、协议层、状态层和 UI 插件层。

- 连接层：封装 SSE/WebSocket/HTTP，支持断线重连和多路复用。
- 协议层：统一处理 Prompt 模板、Function Calling 协议及多模态数据解析。
- 状态层：管理对话上下文（Context）、插件状态及内存（Memory）持久化。
- UI 层：基于组件库构建可插拔的 Chat 容器、工具卡片及交互挂件。
- 监控层：全链路跟踪 Token 消耗、响应耗时及用户满意度（点赞/踩）。

常见坑：

- 架构设计过于耦合，导致更换 LLM 供应商时需要重构大量代码。
- 忽略了跨端（Web/H5/小程序）的兼容性需求。

追问：

- 如何实现 Agent 的“记忆”功能，让它记得三分钟前用户的提问？

2. 在 AI 时代，你认为前端工程师的核心竞争力发生了什么变化？

难度：★★

考点：行业洞察、职业思考

答案要点：

前端的核心竞争力正在从“如何实现 UI”转向“如何定义交互”以及“如何编排 AI 能力”。

- 交互定义：从确定性的点击转向不确定性的自然语言交互（LUI）。
- 模型编排：理解 LLM 的边界，擅长通过 Prompt 和 Tool Use 解决复杂业务问题。
- 性能极致：在流式输出、长文本渲染等新场景下持续优化用户体验。
- 全栈化：更深入地参与到 RAG、向量数据库等后端领域，实现端到端的 AI 体验。

常见坑：

- 陷入“调包侠”的误区，忽视了底层工程化和基础技术的积累。
- 对 AI 技术持排斥态度，不愿尝试新的开发范式。

追问：

- 你最近在关注 AI 领域的哪些前沿技术？

3. 如何衡量一个 AI Agent 前端产品的质量？你会关注哪些核心指标？

难度：★★★

考点：数据驱动、质量保障

答案要点：

除了传统的性能指标，更要关注 AI 特有的交互指标。

- TTFT（首字响应时间）：衡量用户等待感的核心指标。
- 消息生成成功率：评估链路稳定性。
- 交互转化率：用户点击 AI 推荐商品的比例。
- 纠错率：用户手动修改 AI 生成内容的频率。
- 资源占用：AI 组件对直播间整体帧率（FPS）的影响。

常见坑：

- 只关注技术指标，忽略了业务价值指标。
- 统计数据没有区分模型耗时和网络耗时。

追问：

- 如果 TTFT 突然升高，你会从哪些维度去排查问题？

4. 聊聊你在项目中遇到的最难的一个 AI 相关技术挑战，你是如何解决的？

难度：★★★

考点：问题解决能力、技术深度

答案要点：

（此题为开放性回答，需结合实际经验）

- 场景描述：清晰介绍业务背景和技术痛点（如：流式输出下的复杂图表渲染）。
- 方案对比：列举尝试过的几种方案，并说明优缺点。
- 最终决策：解释为什么选择当前的方案（如：采用自定义 DSL 解决模型输出不稳定的问题）。
- 结果产出：通过数据或用户反馈证明方案的有效性。

常见坑：

- 描述过于笼统，没有体现出“难”在哪里。
- 解决方案缺乏逻辑性，听起来像是在堆砌技术名词。

追问：

- 如果现在让你重新做一遍，你会有什么改进？

5. AI 生成的内容具有不确定性，前端如何通过设计手段降低这种不确定性带来的负面体验？

难度：★★

考点：产品思维、交互设计

答案要点：

通过预设引导、结构化约束和反馈机制来增强交互的可控性。

- 引导建议：在输入框上方提供快捷指令（Quick Replies），规范用户输入。
- 结构化输出：强制要求模型返回 JSON，前端通过 UI 卡片（Card）展示，而非纯文本。
- 进度反馈：对耗时操作展示分步进度（如：正在搜索、正在总结、正在出图）。
- 容错编辑：允许用户对 AI 生成的结果进行二次微调，并反馈给模型学习。

常见坑：

- 完全依赖模型的自由发挥，导致 UI 布局频繁崩坏。
- 缺乏对模型“幻觉”的提示。

追问：

- 如何在前端实现一个“对 AI 结果不满意”的反馈闭环？